
Banco de dados

SQLite, DBeaver, ORMs e Prisma

Sumário

BANCO DE DADOS

Conceito de banco de dados relacional

Introdução ao SQLite

Conhecendo ferramentas SGBD

DBeaver

ORM

Conceito · Usabilidade · Cuidados

PRISMA

Conhecendo o Prisma ORM

Instalando o Prisma ORM no nosso projeto

Configurando conexão com banco de dados através do Prisma

Migrations

Cuidados ao desenvolver (npx migrate)

Relembrando

Banco de dados relacional

Estruturado em tabelas

Usam chaves primárias e estrangeiras para relacionar tabelas

Utilizam SQL

Possuem schema fixo (campos definidos)

MySQL, Postgres, SQLite, MariaDB, Oracle, SQL Server...

Banco de dados não relacional

Podem seguir esquema de arquivos, colunar, chave-valor, grafos...

Permitem schemas dinâmicos (campos variam)

Otimizado para grandes volumes de leitura e escrita

Cassandra, DynamoDB, MongoDB, Redis, Neo4j

SQLite

Banco de dados baseado em arquivo

Fácil manipulação

Fácil sintaxe (SQL)

OpenSource (código aberto)

```
CRUD-DB
  migrations
  node_modules
  dev.db
  package-lock.json
  package.json
  schema.prisma
  server.js
```

O banco inteiro é um arquivo dentro do projeto: **dev.db**

Instalando o SQLite

sqlite.org

LATEST RELEASE

Version 3.50.4 (2025-07-30)

Download

Prior Releases

Usando o WINGET

INSTALANDO

- 1 Botão Windows
- 2 Powershell
- 3 Digitar:

```
winget install SQLite.SQLite
```

VERIFICANDO A VERSÃO

- 1 Feche o terminal e abra um novo
- 2 Digite o comando:

```
sqlite3 --version
```

Caso falhe o WINGET: o manual de instalação de forma manual está no GitHub, nos arquivos auxiliares.

SGBD

Sistema Gerenciador de Banco de Dados

Intermediário entre o banco de dados e o usuário que o utiliza.

Gerencia a criação, acesso, manipulação, segurança e organização dos dados.

| *Interface bonitinha pra você usar o banco de dados*

Instalando o DBeaver

<https://dbeaver.io/download/>

DOWNLOAD

DBeaver Community 25.2.0

Lançado em 31 de agosto de 2025. Gratuito e open source.

Windows: instalador (installer) ou arquivo zip.

Importante!

Na instalação do DBeaver, na tela **Escolher componentes**:

DBeaver Community	Marcado (obrigatório)
Include Java	Marcado
Reset Settings	Desmarcado
Associate SQL files	Desmarcado
Associate SQLite database files	Marcado (não esqueça)

Conectando ao banco com o DBeaver

- 01 Nova conexão
- 02 Escolher o tipo de banco (SQLite)
- 03 Selecionar um arquivo de banco de dados (vamos criar um como exemplo)

CASO SE DEPAREM COM A JANELA "CONFIGURAÇÕES DE DRIVER"

Basta clicar em "**Download**" que os pacotes serão baixados.

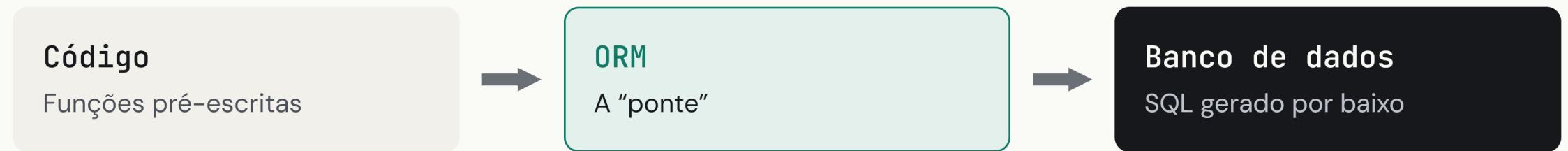
02

ORMs

O que é um ORM

Object-Relational Mapping (Mapeamento objeto-relacional)

É uma espécie de “ponte” entre a programação em si e o banco de dados.



Permite que você utilize funções pré-escritas ao invés de escrever um SQL puro.

Para que serve

- 01 Abstração de SQL: você escreve código, programa, ao invés de SQL
 - 02 Possuem proteção automática contra *sql injection*
 - 03 Desenvolvimento mais rápido
 - 04 Facilita a mudança entre diferentes bancos de dados
-

Cuidados e desvantagens

Performance inferior

Pode gerar queries demoradas

Queries complexas

Algumas consultas avançadas (muitos joins) podem acabar se tornando mais complexas usando ORM do que usando SQL

Curva de aprendizagem

Cada ORM tem sua própria linguagem e suas próprias funções

Dependência

Seu código fica acoplado ao ORM

Como eu sei o recurso que cada um tem? Preciso decorar tudo?

Não precisa decorar.

01

Documentação oficial

02

Bookmarks

03

Praticar principais
métodos (CRUD)

03

Prisma ORM

Estrutura básica

CRUD-DB

- migrations
- node_modules
- dev.db
- package-lock.json
- package.json
- schema.prisma
- server.js

schema.prisma

O arquivo principal do Prisma, na raiz do projeto, ao lado do **dev.db**.

schema.prisma

CONEXÃO E CLIENT

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "sqlite"  
  url      = "file:./dev.db"  
}
```

TABELA

```
model User {  
  id    Int    @id @default(autoincrement())  
  nome  String  
  email String  @unique  
  
  @@map("usuarios")  
}
```

Conhecendo o Prisma (documentação)

Prisma Documentation

Trabalhar com o Prisma dá a você um ORM TypeScript de primeira linha, um sistema declarativo de migrations e um banco de dados com tudo para começar.

Experimente o que o Prisma oferece com um único comando:

```
$ npx prisma init --db
```

Vantagens do Prisma

01

Type-safe

Gera um *client* específico, do tipo do schema

02

Queries otimizadas

03

Schema declarativo

04

Boa experiência de desenvolvimento

Sessões importantes

1	Getting started	Tutorial passo a passo
2	Schema reference	Referência da sintaxe
3	Client API	Como usar o Prisma Client
4	Migrate	Sistema de migrações
5	Examples	Projetos prontos para estudar

schema.prisma

Arquivo principal do Prisma

DEFINE

Tabelas

Relacionamentos

Índices

Configurações do banco de dados

Configurações do client

Instalando o Prisma

```
BASH
```

```
# Prisma CLI (desenvolvimento)
```

```
npm install prisma --save-dev
```

```
# Prisma Client (produção)
```

```
npm install @prisma/client
```

Inicializando o Prisma no projeto

BASH

```
npx prisma init
```

ESSE COMANDO

01

Cria a pasta "prisma"

02

Cria o arquivo
schema.prisma

03

Cria um arquivo .env (vamos
falar sobre isso)

04

Migrations

O que são migrations

Scripts de **versionamento de banco de dados**, que permitem:

01

Evoluir o schema (criar novos campos, novas tabelas, etc.) de forma controlada

02

Sincronizar mudanças entre ambientes

03

Manter histórico de alterações

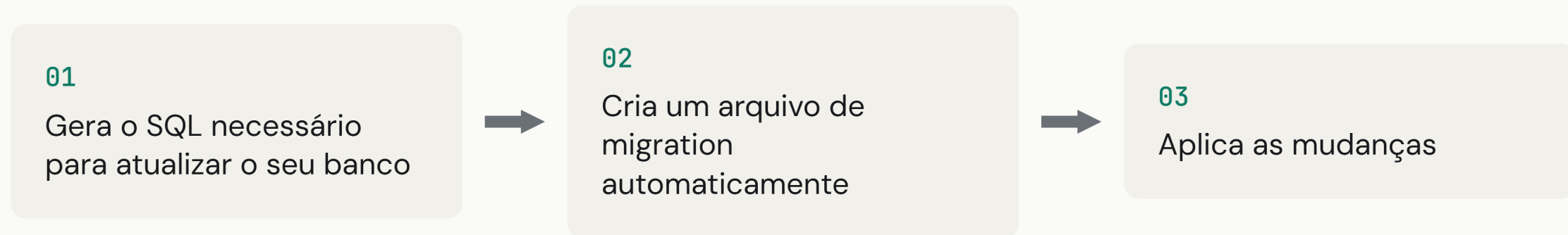
04

Reverter mudanças quando necessário

Como aplicar mudanças em tabelas

O Prisma compara o **schema.prisma** com o estado do seu banco atual.

Quando ele encontra algo que mudou, realiza o seguinte processo:



Logo: para atualizar um banco, basta atualizar o arquivo *schema.prisma*.

Executando migrations

BASH

```
npx prisma migrate dev
```

Atenção

UPGRADE

O Prisma realiza o upgrade do banco de dados automaticamente, através dos migrations.

DOWNGRADE

Mas não possui downgrade: é preciso realizar essa ação manualmente, seja com um SQL “manual” ou algum outro procedimento.

PRÁTICA

Vamos tentar na prática

```
npx prisma init
```

```
schema.prisma
```

```
npx prisma migrate dev
```